

```
while(abs(sorted_cabinet[guess] - looking_for) > 0.0001):
    if(sorted_cabinet[guess] > looking_for):
        upperbound = guess
        guess = math.floor((guess + lowerbound)/2)
    if(sorted_cabinet[guess] < looking_for):
        lowerbound = guess
        guess = math.floor((guess + upperbound)/2)
return(guess)

print(binarysearch(sortedcabinet,8))
```

[Listing 4-7:](#) *An implementation of binary search*

The final output of this code tells us that the number 8 is at position 7 in our `sorted_cabinet`. This is correct (remember that the index of Python lists starts at 0). This strategy of guessing in a way that eliminates half of the remaining possibilities is useful in many domains. For example, it's the basis for the most efficient strategy on average in the formerly popular board game *Guess Who*. It's also the best way (in theory) to look words up in a large, unfamiliar dictionary.

Applications of Binary Search

Besides guessing games and word lookups, binary search is used in a few other domains. For example, we can use the idea of binary search when debugging code. Suppose that we have written some code that doesn't work, but we aren't sure which part is faulty. We can use a binary search strategy to find the problem. We split the code in half and run both halves separately. Whichever half doesn't run properly is the half where the problem lies. Again, we split the problematic part in half, and test each half to further narrow down the possibilities until we find the offending line of code. A similar idea is implemented in the popular code version-control software Git as `git bisect` (although `gitbisect` iterates through temporally separated versions of the code rather than through lines in one version).

Another application of binary search is inverting a mathematical function. For example, imagine that we have to write a function that can calculate the arcsin, or inverse sine, of a given number. In only a few lines, we can write a function that will call our `binarysearch()` function to get the right answer. To start, we need to define a domain; these are the values that